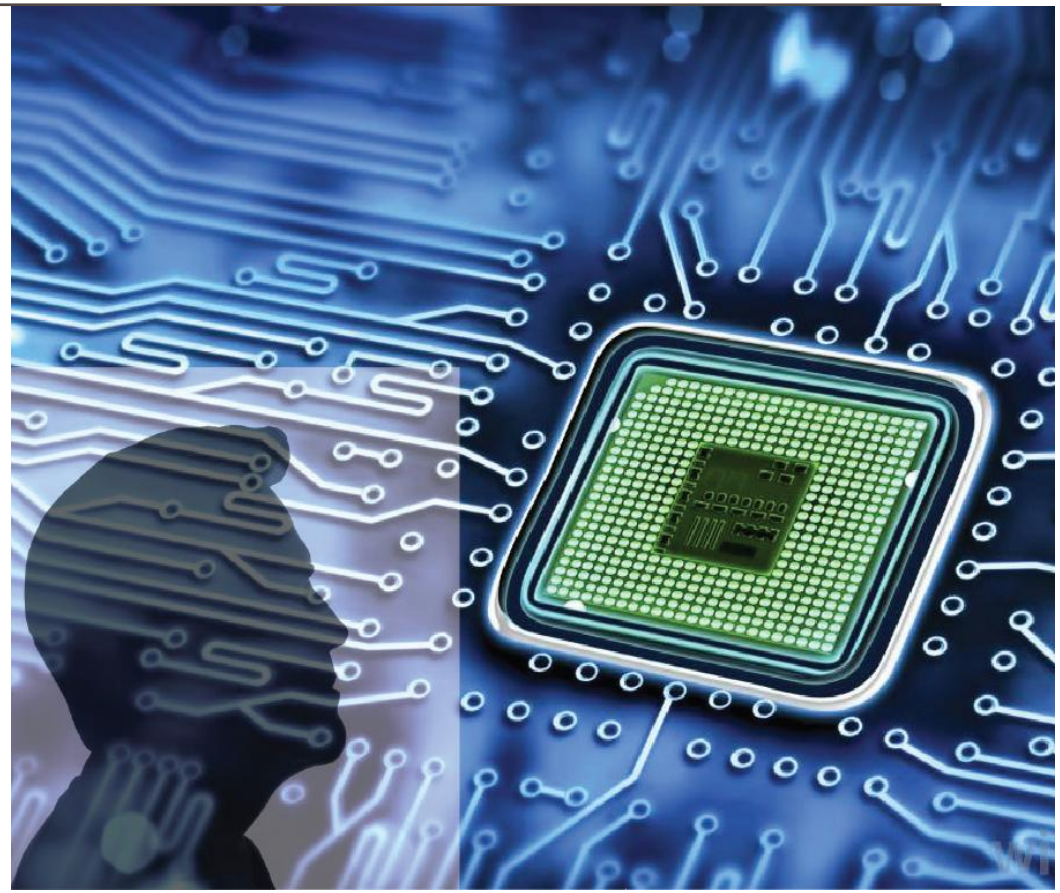
ADVANCED DIGITAL DESIGN AND VERIFICATION

Week-1, Lecture-1
Introduction



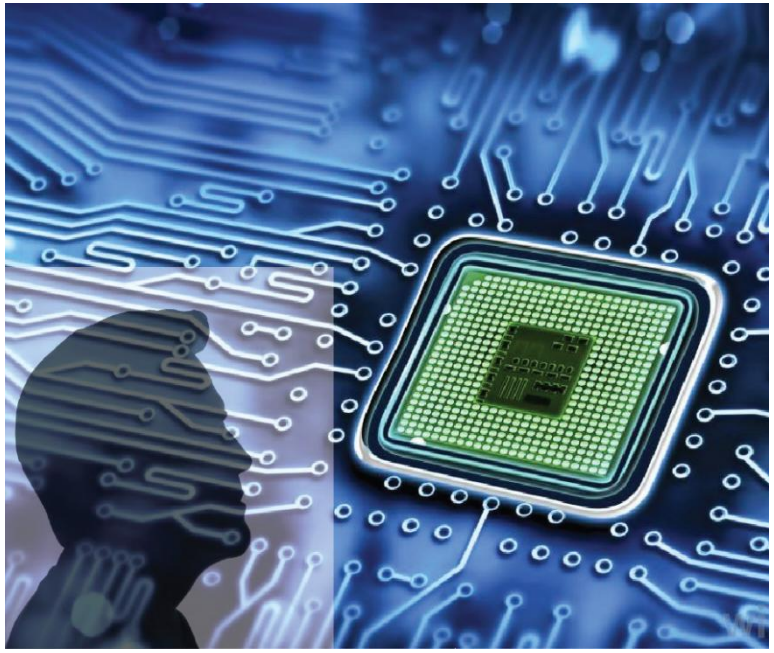
Sneh Saurabh
12th August, 2025

About the Course

Advanced Digital Design and Verification (ADDV): Objective

- Develop a basic understanding of the methods, tools, and technologies involved in designing and verifying a **complex digital system**

- Gain a practical understanding of the **advanced design and verification methodologies**



ADDV: Expected outcome

- Clearly understand and apply various **tasks** involved in designing complex digital systems at the **pre-RTL level**
- Can evaluate various **trade-offs** that need to be made at various stages of **pre-RTL design**
- Clearly understand and apply various **advanced functional verification techniques**
- Can design and verify a **digital system at various abstraction levels** using computer-aided design (CAD) tools.

ADDV: Pre-requisites

- VLSI Design Flow (ECE313/ECE513)

- C++ or Concepts of Object-oriented Programming

- Course has significant programming component
  - SystemC: Essentially C++
  - SystemVerilog: Verilog + OOP

```
if (you dislike or fear programming) {
    if (you can live this way) {
        Do not take this course.
    } else {
        Take this course.
        if (effort is high) {
            You will see the benefits. Enjoy!
        } else {
            You might fail.
        }
    }
} else {
    Take this course; Enjoy;
}
```

ADVV course will NOT fulfil the core course requirement of MTech ECE (VLSI)

ADDV: Where to apply the learning?

- Will be used extensively in the semiconductor industry at the system level
- VLSI, CAD, Semiconductors, Circuit Design, FPGA, Embedded Systems, Computer Architecture ….

- A lot of manual effort is still needed at the system-level
  - ➢ Many opportunities
- Growing area: futuristic
- Innovation

ADDV: Course Content

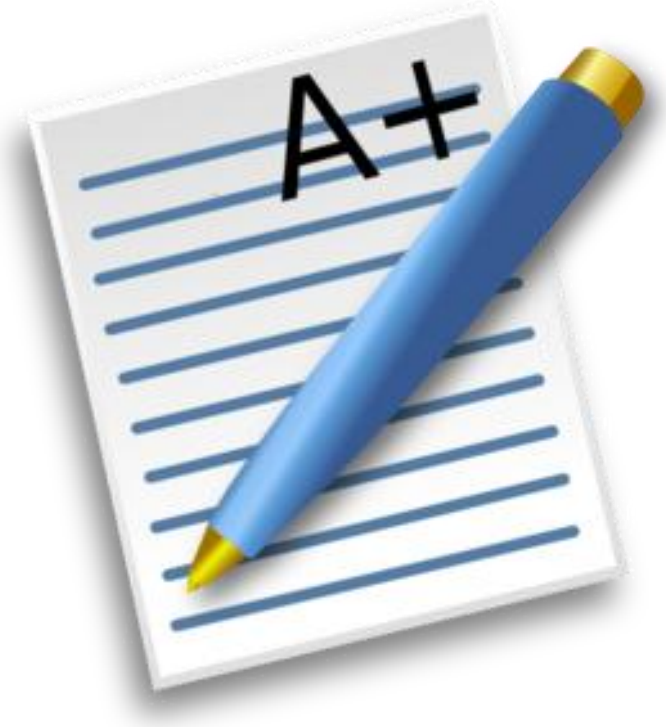
Week No.	Topic
1	Introduction; Digital Systems: Hardware and Software components; Software Development Process; Hardware Design and Verification Flow; Manufacturing and Test
2	Electronic System Level (ESL) Design: SystemC Models, Transaction-level Models (TLM), Virtual Platforms and Platform-based Design, Applications
3	Hardware-software co-design: models, design space exploration, partitioning, co-simulation
4	High-level Synthesis: Flow, Tasks, Control Data Flow Graph (CDFG), Optimizations
5	High-level Synthesis: Scheduling, Resource Allocation, Handling loops, arrays and functions, Pipelining
6	System-on-Chip (SoC) Design Methodology: IPs, IP Packaging, IP-XACT, IP Assembly, Generating RTL, Challenges
7	Introduction to SystemVerilog: Data Types, Improvements over Verilog for Synthesis and Simulation, Object-oriented Programming Constructs, Interfaces, Storage Elements, Inter-process Communications
8	Advanced Concepts on Simulation: compiled-simulators, random vector-based verification, constrained-random verification, coverage-driven verification
9	Advanced Concepts on Formal Verification: fundamentals of Property Checking, Assertion-based Verification, Bounded model Checking
10	Universal Verification Methodology (UVM); Scenario-based Software-driven Testing; Portable Test and Stimulus Standard (PSS); Hardware-assisted Verification
11	Clock Domain Crossing (CDC): Problem, Solutions, Verification; Reset Domain Crossing (RDC)
12	System-level Power Management and Optimizations: Dynamic Power Management, Policies, Implementation, Voltage Scaling, Software-level power management
13	Power Intent: Unified Power Format (UPF) and design flows

ADDV: Google classroom

- Google classroom
  - Announcements
  - Sharing lecture slides, assignments etc.
    - Slides will be password protected. The password is announced in the classroom. If you do not know, ask your classmate
  - Submission of assignments
- Link: https://classroom.google.com/u/0/c/NzkxNjgyMTg3NjQ4
- *agadoic7*

ADDV: Evaluation Criteria

Type of Evaluation	Contribution in Grade
Weekly Assignment	25 %
Mid-sem	30 %
End-sem	45 %



ADDV: Assignments

- Assignment in Pairs (You need to form a group)
- 4-5 assignments will be given (all will be considered)
- All assignments will be scaled to 10 Marks
- Total obtained marks will be scaled appropriately to make the contribution of Assignments to 25%

- Assignment will be uploaded on Google Classroom
- Typically, one week time to complete assignment on Google Classroom
  - Do not request for extension, ideally you should submit well before deadline
- Strict Plagiarism check will be done [Beware of this !!! No excuse will be entertained]

Mid-sem/End-sem Exams

- Questions will be asked from Assignment also
- Closed book

VDF: EDA Server

- Tools may be run on EDA Server: UNIX
- Login for all the students must exist (because of VDF course)
  - ➢ If not, login will be created on this server
- Demo for assignment must be kept ready in your account
- You may be asked to give Demo in certain cases

References

- References will be provided during lectures
- Saurabh, S. (2023). Introduction to VLSI Design Flow. Cambridge: *Cambridge University Press*.
- Black, David C., and Jack Donovan, eds. SystemC: From the ground up. Boston, *MA: Springer US, 2004*.
- Martin, Grant, Brian Bailey, and Andrew Piziali. ESL design and verification: a prescription for electronic system level methodology. *Elsevier*, 2010.
- Scheffer, Louis, Luciano Lavagno, and Grant Martin, eds. EDA for IC system design, verification, and testing. *CRC press*, 2018.
- Spear, Chris. SystemVerilog for verification: a guide to learning the testbench language features. *Springer Science & Business Media*, 2008.
- Mehta, Ashok B. SystemVerilog Assertions and Functional Coverage. *Springer International Publishing*
- Micheli, Giovanni De. Synthesis and optimization of digital circuits. *McGraw-Hill Higher Education*, 1994.

Policies

ADDV: Attendance Policy

- Student with less than 30% attendance [less than 8 classes] will be awarded FAIL grade ***at the discretion of the instructor*** [even when the total marks is above the PASS grade]
- Do not mark proxy attendance

ATTENTION: DO NOT Register for the course if you do not plan to attend the classes regularly (for ANY reason!)

ADDV: Policies (1)

- Be punctual to class: up to 5 minutes late is tolerated, beyond that please do not come

- Cell phones, Laptops, IPODs, MP3, players or other portable electronic devices of any kind NOT allowed in classroom (should be switched off)

ADDV: Policies (1)

- Be punctual in submission of Assignments and Projects
  - ➢ Each day of delay will attract 20 % penalty and after 5 days delay no more submission

Illustration: Day of submission is 20th August

- Submission is on-time if it is done by 12:00 midnight of 20th August
- Submitted at 8:00 am 21th August: Penalty of 20%, If marks obtained was 20, it will change to 16 marks
- Submitted at 1:00 am 25th August: Penalty of 100%, If marks obtained was 20, it will change to 0 marks

ADDV: Plagiarism

- Academic dishonesty/plagiarism/cheating
  - IIITD rules will apply
  - Academic dishonesty policies of IIIT Delhi apply. https://www.iiitd.ac.in/sites/default/files/docs/education/AcademicDishonesty.pdf
  - Checks will be in place to detect copying/plagiarism/cheating

ADDV: Questions/Doubts

1. Class: Many may have the same questions
2. Just After Class
3. Office Hours
4. During revision
5. No last minute query for assignment submission
   - Clear all your doubts at least 2 days before submission

ADDV: Office Hours

Lecture:

- Venue: C210
- Tuesday: 3:00-4:30 pm
- Thursday: 3:00-4:30 pm

Instructor:

12:30 pm - 1:30 pm: Wednesday, B-608

TAs

- Varun

ADDV: Expectations

- Revise last day lecture before coming
  - Revision will be done at the start of every lecture

- Ask questions and be active participants
- Course expected to be at pace so that **EVERYONE** is able to follow
- Give feedback

Introduction

Electronic System

What is a system?

- A "system" is the **combination of elements** that function together to produce the capability required to meet a need [1].

[1] https://www.nasa.gov/reference/2-0-fundamentals-of-systems-engineering/ Fundamentals of system engineering

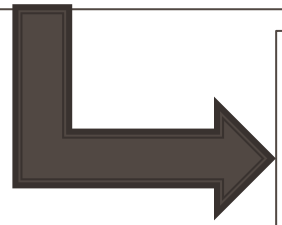
Elements: can include hardware, software, equipment, facilities, personnel, and processes.

Electronic systems:

- Systems designed to sense, process, and manipulate **electronic signals**
- Example: mobile, compute, robot, television, EEG machine, etc.

The CDC 6600, considered to be the first supercomputer (1964). https://en.wikipedia.org/wiki/CDC_6600

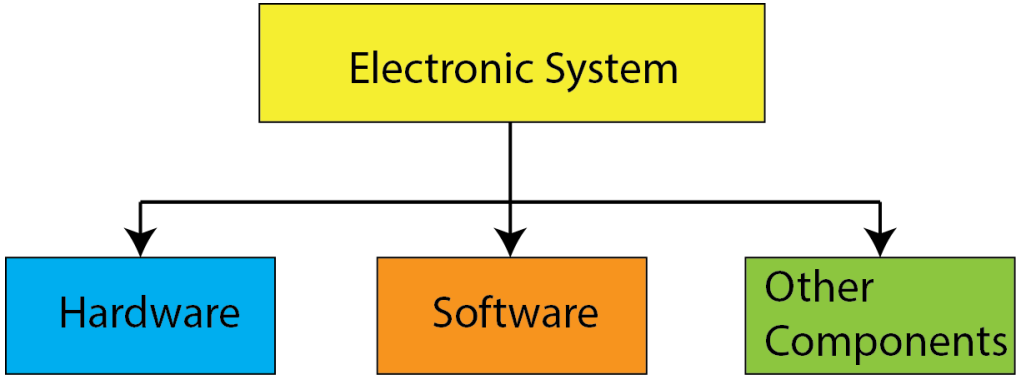
Introduction



System Components

Components of Electronic Systems

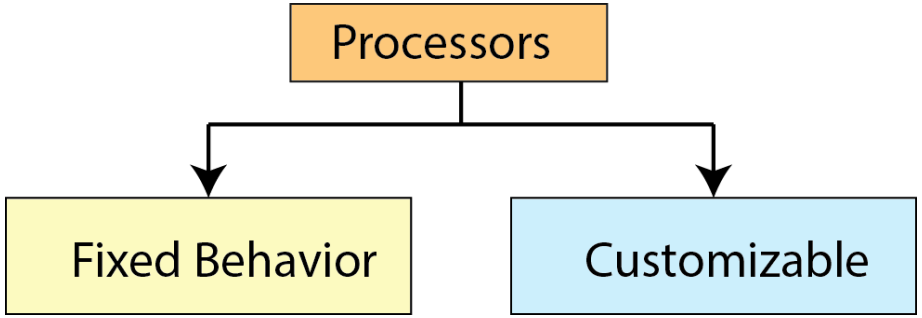
- System can consist of a large number of subsystems created with diverse technologies
- The functionality of the system is richer than individual systems
- Subsystems can be grouped into three categories



- **Hardware (electronic):** processing, storage (memory), sensing, communication, etc.
- **Software:** real-time and non-realtime SW, operating system, DSP algorithms, user interface, etc.
- **Other components:** packaging, printed circuit boards, moving parts like hinges, keyboards, etc.

Hardware: Processors Type

Classification based on ability to customize

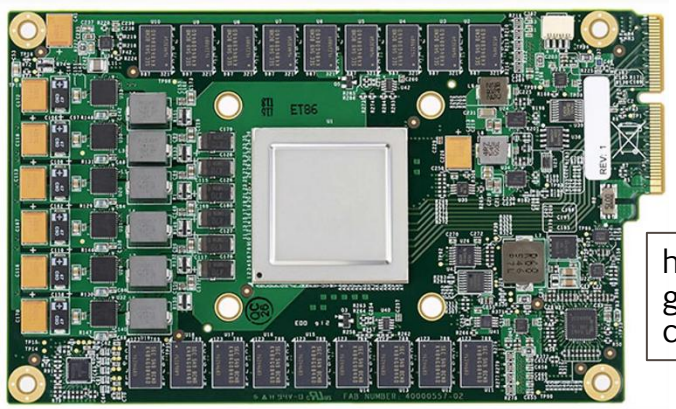


Hardware: Fixed Behavior Processors

Fixed behavior: instruction set, architecture

- **General purpose**: designed to meet general computing requirements
  - Example: Intel or AMD x86-class, ARM
- **Special purpose**: optimized to perform specific task more efficiently (power, speed, etc.)
  - DSP processor (eg. TI's TMS320VC541)
  - Image processor (eg. Qualcomm Spectra)
  - Neural Processor Unit (eg. NXP's eIQ Neutron NPU)
  - Tensor Processing Units (TPUs), Graphical Processing Unit (GPUs)

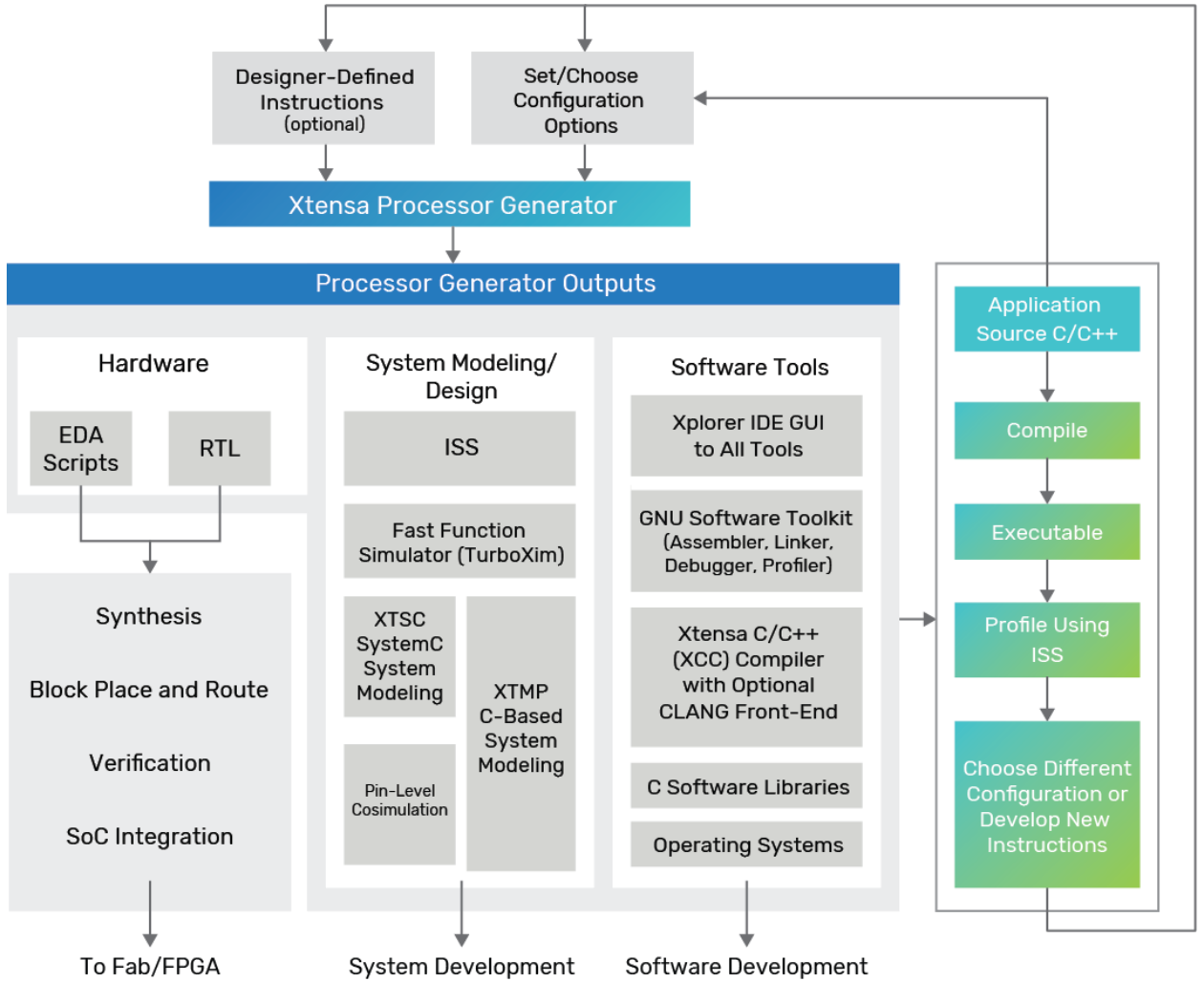
https://www.intel.com/content/www/us/en/newsroom/news/13th-gen-core-launch.html#gs.cvzdo3



https://spectrum.ieee.org/google-details-tensor-chip-powers

Hardware: Customizable Processors

- **Extensible** and **Configurable Processors**:
  - Application specific instructions set (ASIP): instruction set can be extended to improve PPA (e.g., energy-efficient for low-power audio-video processing)
  - Configurable for cache size, pipelining, other features
- Example: Cadence Tensilica Xtensa processors and Synopsys ARC-V Processor
  - Entire tool chain to support application-specific instructions and configuration



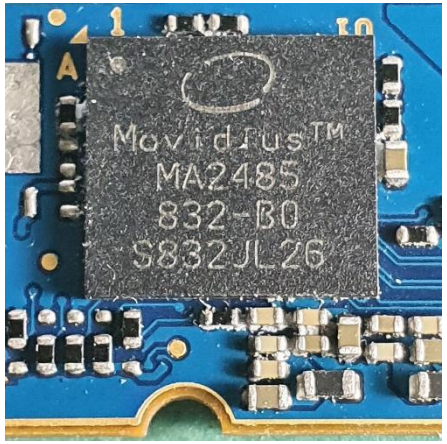
Block diagram of Xtensa LX7 processor architecture
https://www.cadence.com/en_US/home/resources/datasheets/xtensa-lx7-processor-ds.html

Hardware: Coprocessors

- Coprocessors: additional processor that is used to supplement the main processor for some tasks
- Example: floating-point arithmetic, graphics, signal processing, string processing, cryptography, I/O interfacing, artificial neural networks, vision processing

- Optimized for these tasks (not for general purpose computing)
- Allows main processor to have less functionality
  - Products based on solely main processor are less costly
- Over time CPUs have tended to grow to absorb the functionality of the most popular coprocessors.

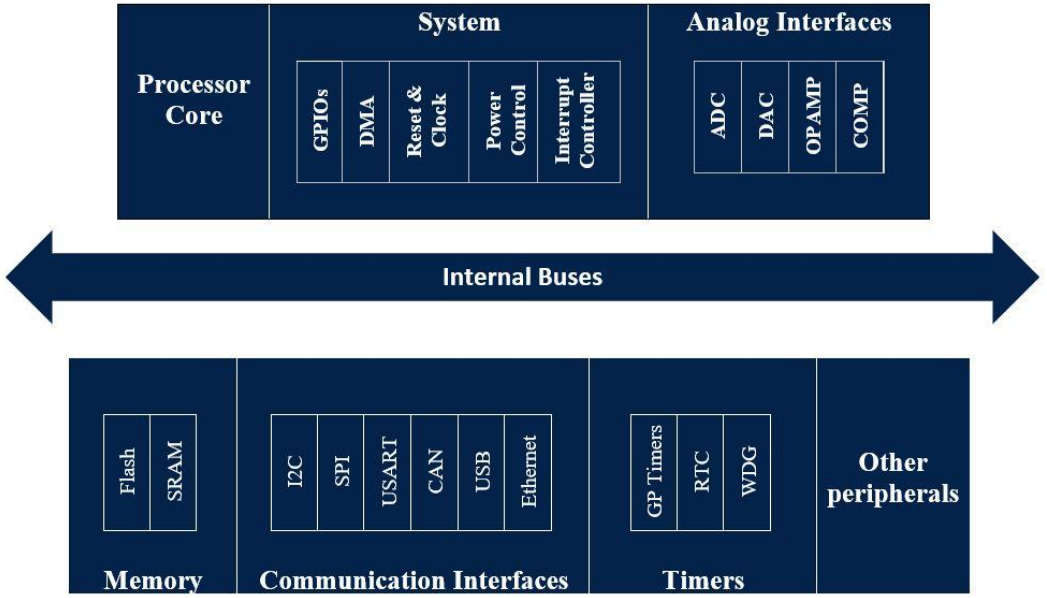
Intel 80386DX CPU with 80387DX math coprocessor
https://en.wikipedia.org/wiki/Coprocessor



Intel's Myriad X VPU (vision processing unit)
https://en.wikipedia.org/wiki/Movidius#Myriad_X

Hardware: Microcontrollers

- Microcontroller: contains one or more CPUs (processor cores) along with memory and programmable input/output peripherals built on a single IC
  - Similar to SoC (though with less sophistication): an SoC can contain microcontroller
- Examples: Microchip's PIC 18F8720, Intel's 8742, STMicro's STM32, etc.
- Often contains A/D converters (to sense signal), programmable interval timers (generate interrupt), etc.
- Designed for embedded applications, automatically controlled system (automobile, robot, toys, medical devices, IoT)
  - Less cost than microprocessor + memory + IO



Basic Layout: https://wiki.st.com/stm32mcu/wiki/STM32StepByStep:STM32MCU_basics

- Contains several general purpose input/output (GPIO) pins
  - GPIO pins (configured as input): often used to read sensors or external signals.
  - GPIO pins (configured as output): drives LEDs, motors, actuators (through external power electronics)

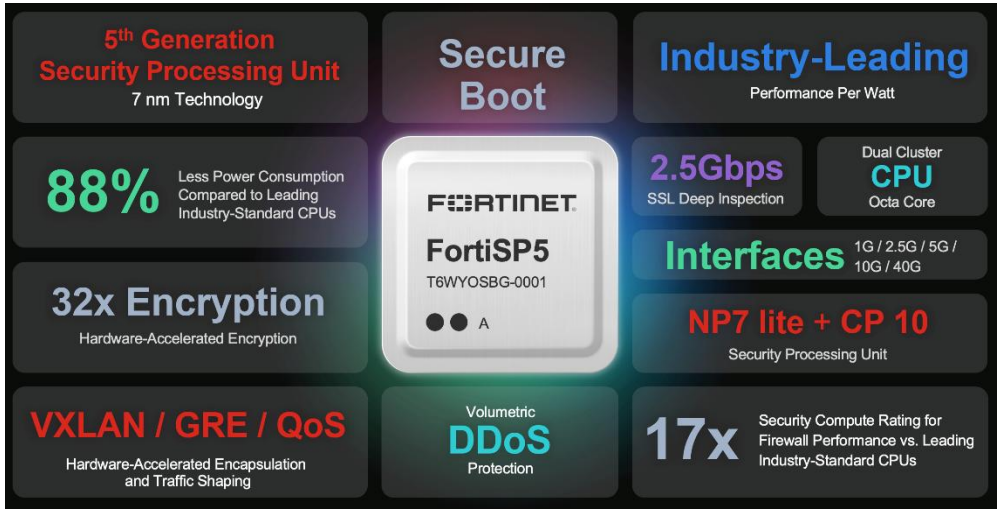
Hardware: ASIC

- ASIC: designed for specific application, rather than for general-purpose use
- Can contain multiple processors, memory, peripherals, and other building blocks
  - Complex ASICs are same as SoCs

Examples: Chips for

- Networking
- Security
- Voice-recording
- Encoding-decoding video, etc.

https://blogs.cisco.com/networking/cisco-rf-asic-named-best-enterprise-wi-fi-network-technology-by-the-wireless-broadband-alliance

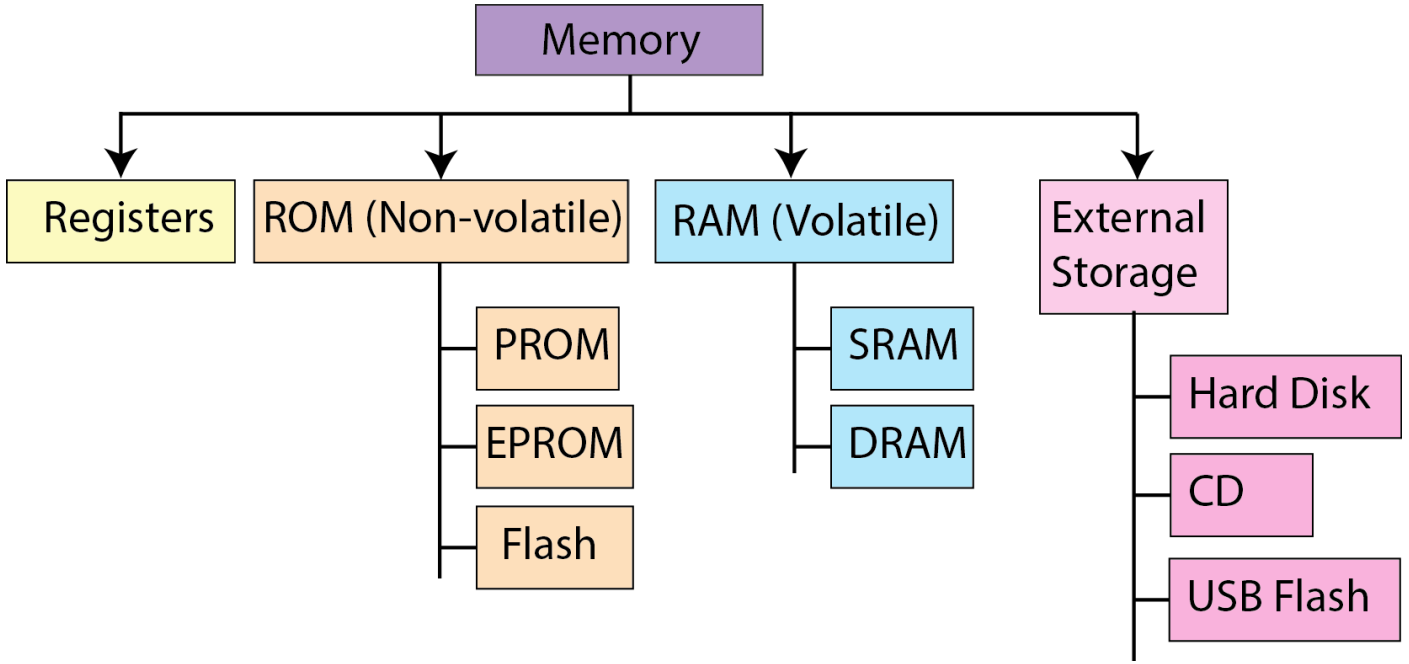


https://www.fortinet.com/corporate/about-us/newsroom/press-releases/2023/fortinet-unveils-new-asic-accelerate-networking-security-convergence-across-network-edges

Hardware Components: Memory

Memories:

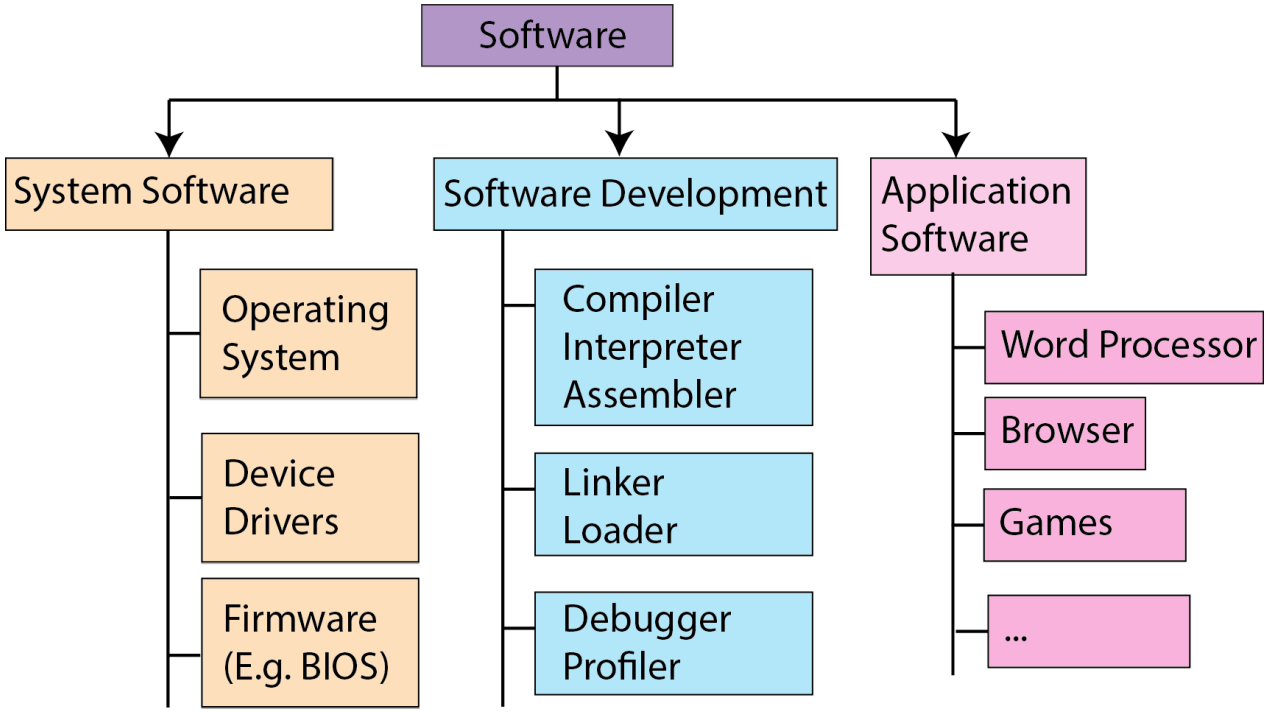
- Can be part of an IC (in processors, coprocessors, microcontrollers, ASICs, SoCs, etc.)
- Can be standalone component that is integrated at the system level



Software Components

Software are of various types:

- **System software:** programs used to start and run computer systems
- **Software tools for software development:** translate code into executables
- **Application software:** programs performing various tasks for the user
  - ➢ Can be general-purpose or specific-purpose

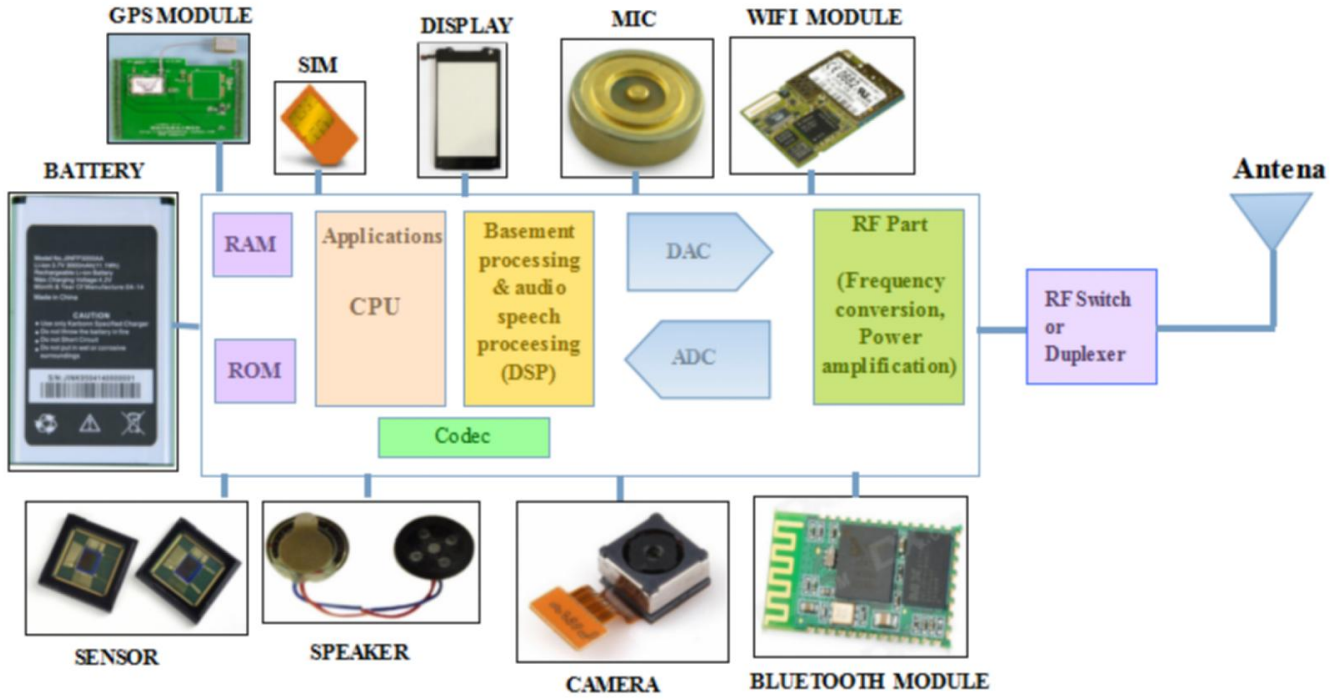


- **Firmware:** machine instructions embedded into hardware devices (in cameras, mobile phones, network cards, printers, routers, and television remotes)

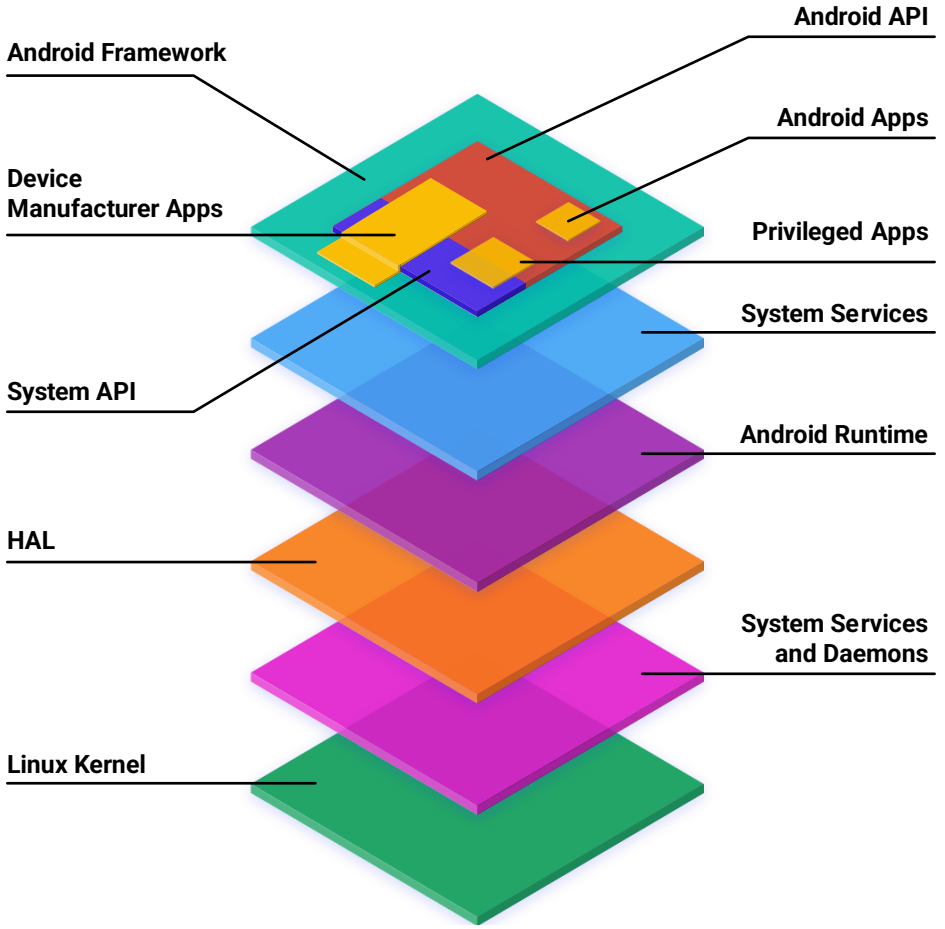
A pair of AMD BIOS chips for a Dell 310 computer from the 1980s. https://en.wikipedia.org/wiki/BIOS

Hardware And Software Components: Example

- Components in a mobile phone

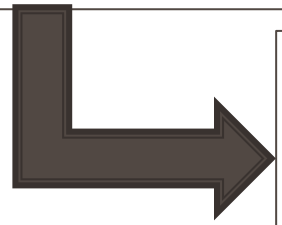


https://www.techplayon.com/mobile-phone-architecture/



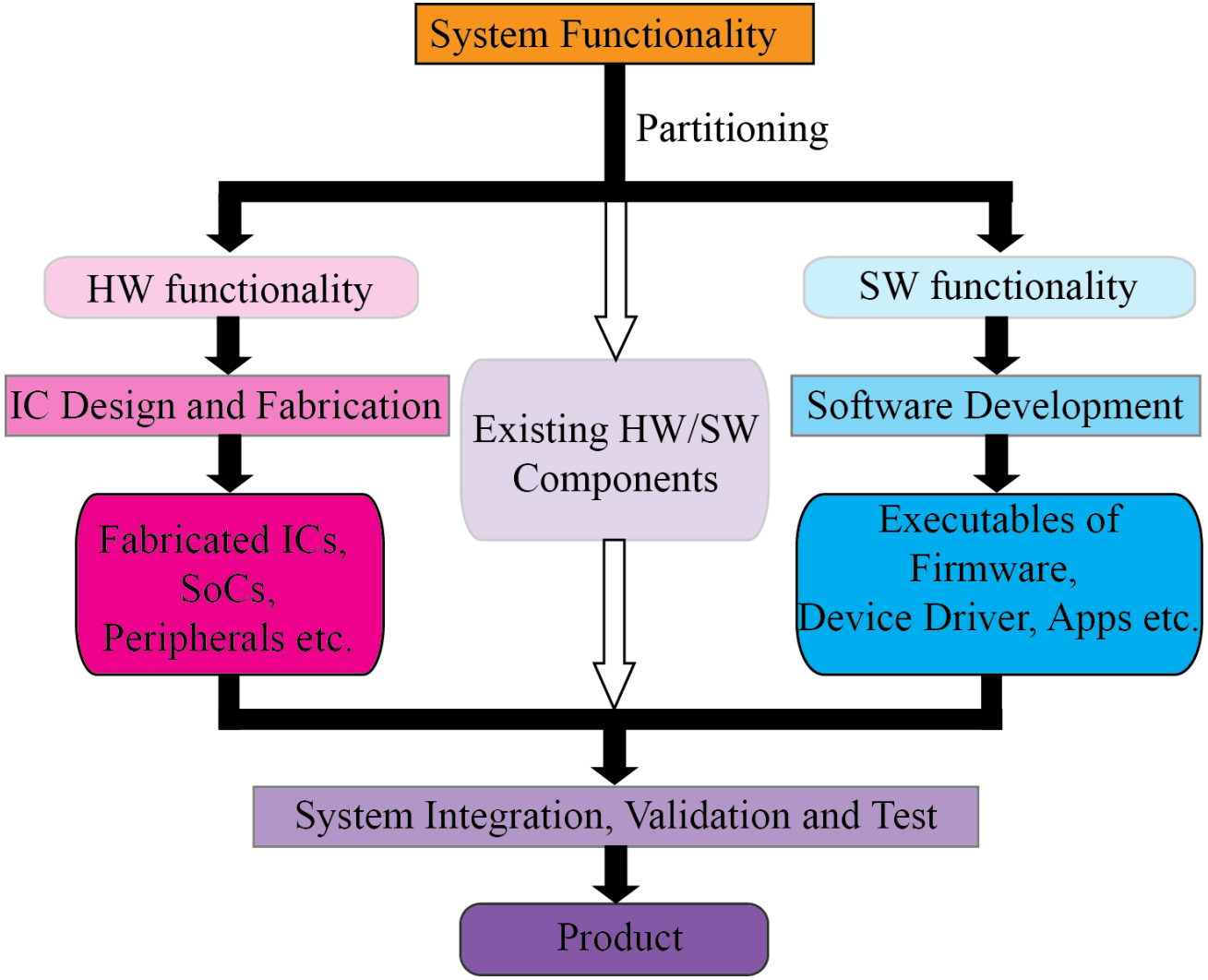
https://source.android.com/docs/core/architecture

Introduction



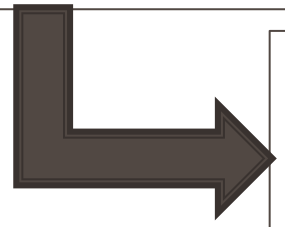
System Implementation

System Implementation: Bird's Eye View



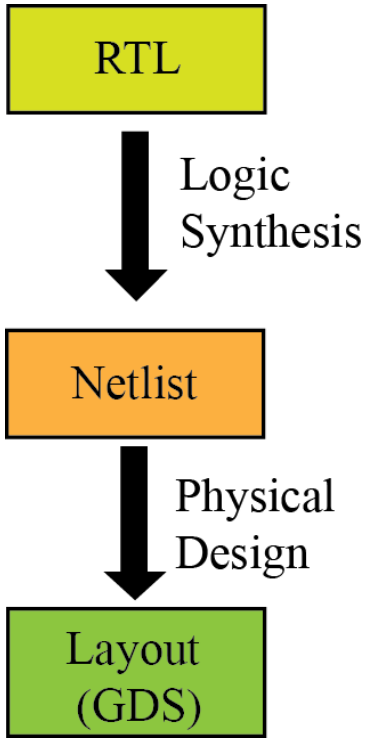
1. Map functionality on to components
2. Build Components
   A. HW design, verification, fabrication
   B. SW Development
3. Integration, validation

Introduction



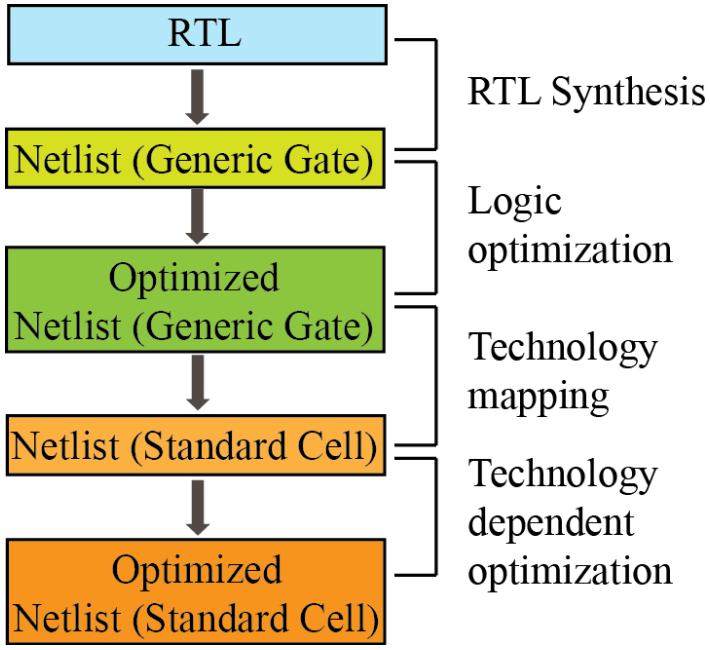
System Implementation: Hardware

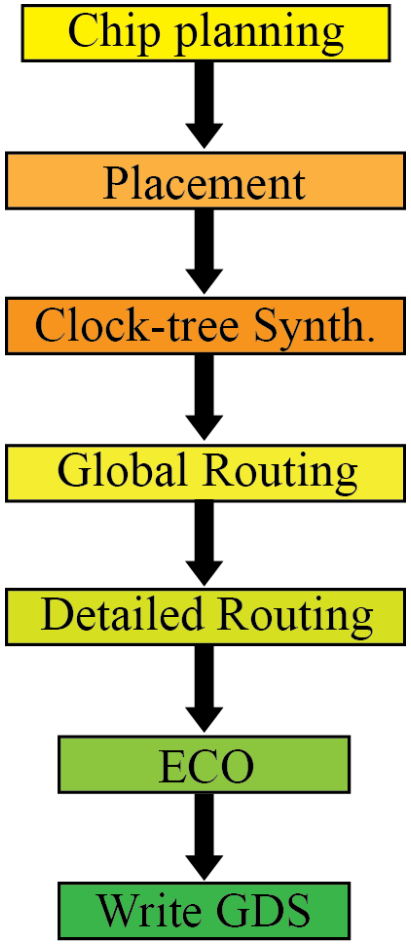
Hardware Design Implementation



- **Logic Synthesis:** RTL is converted to an equivalent netlist

Physical Design: netlist is converted to an equivalent layout

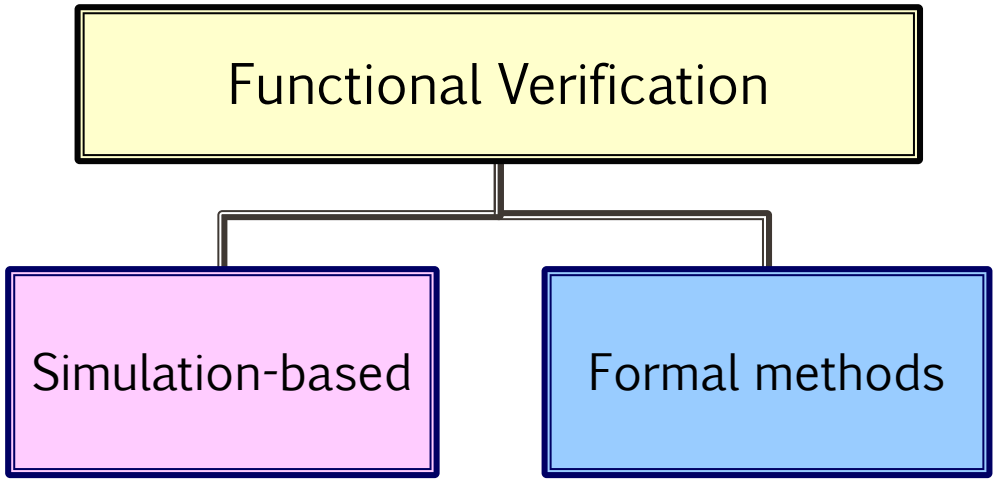




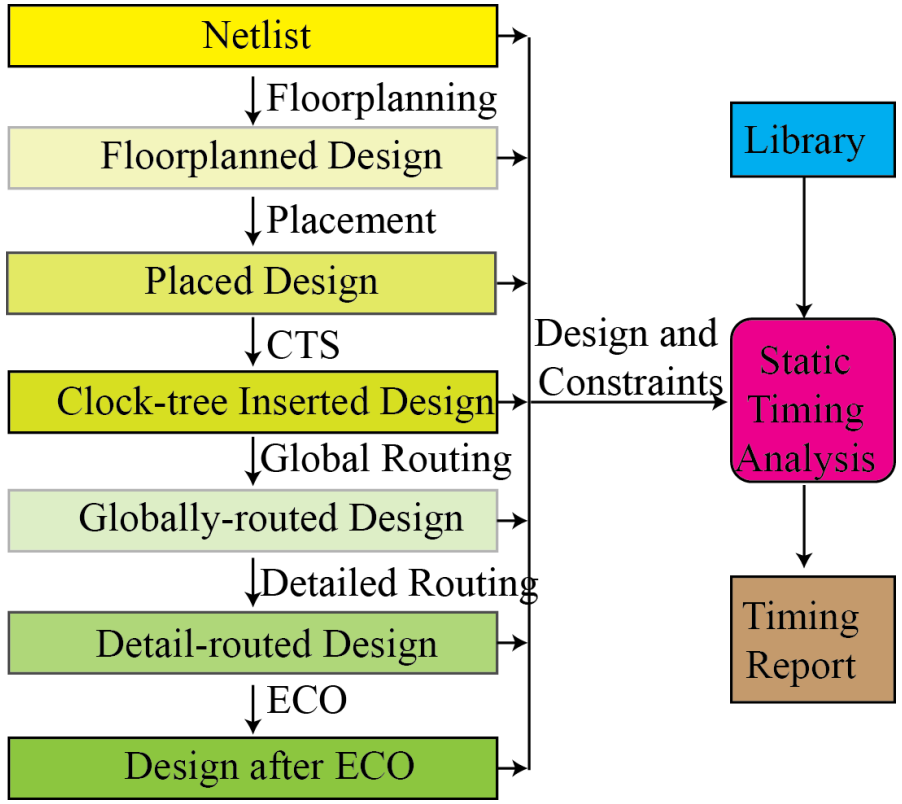
Hardware Design Verification

Verification:

- To ensure that the design works as per given functionality
- Verification is done multiple times throughout a VLSI design flow

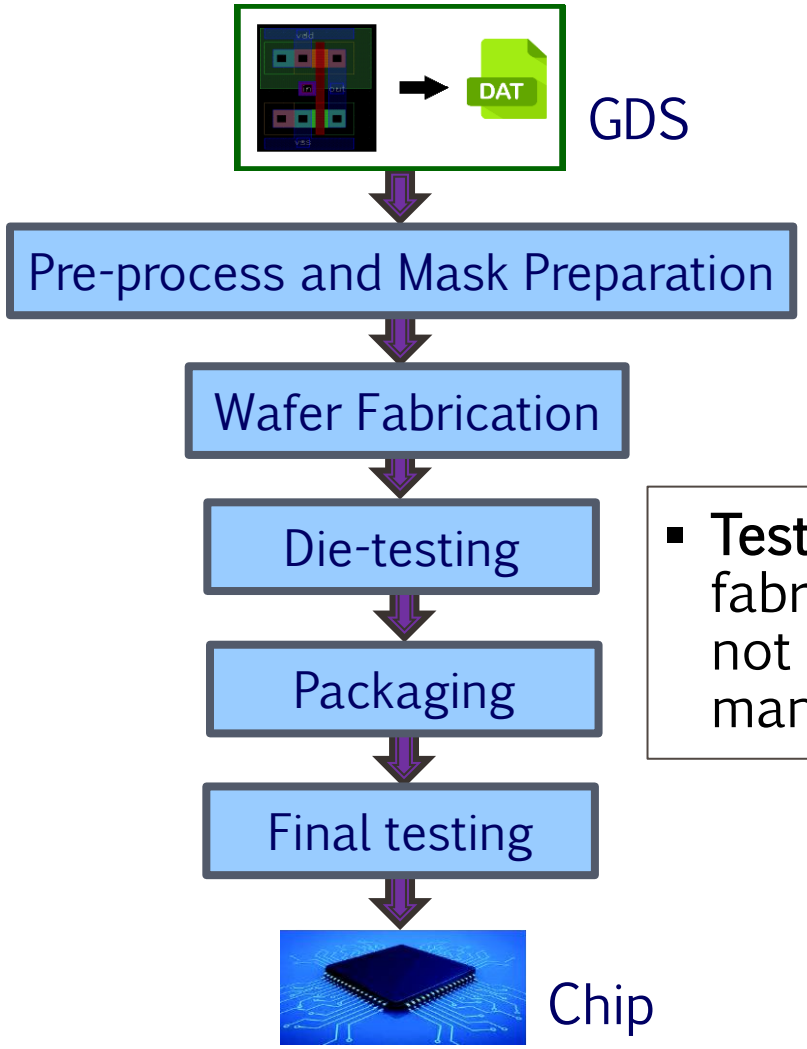


- Equivalence Checking

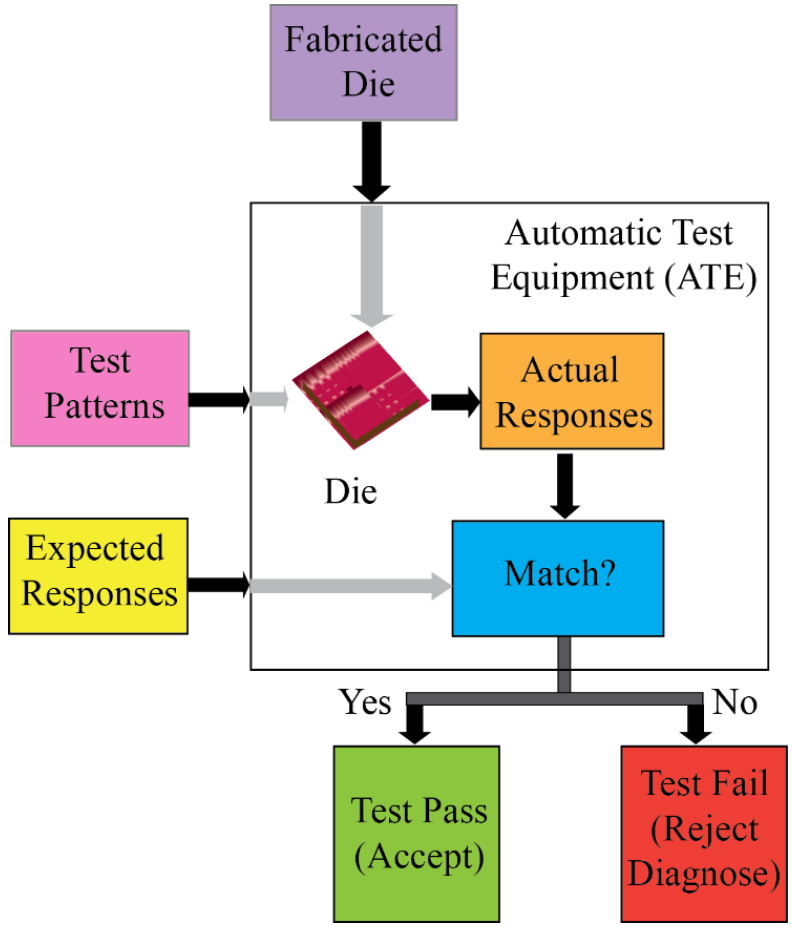


- Physical Verification: DRC, ERC, LVS, etc.
- Signal Integrity, IR drop analysis, etc.
- Signoff checks

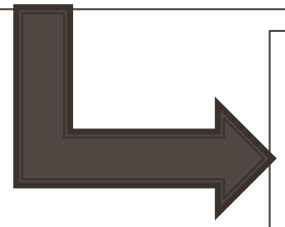
Hardware Manufacturing and Test



- **Testing** : Ensures that fabricated chip does not have any manufacturing defect

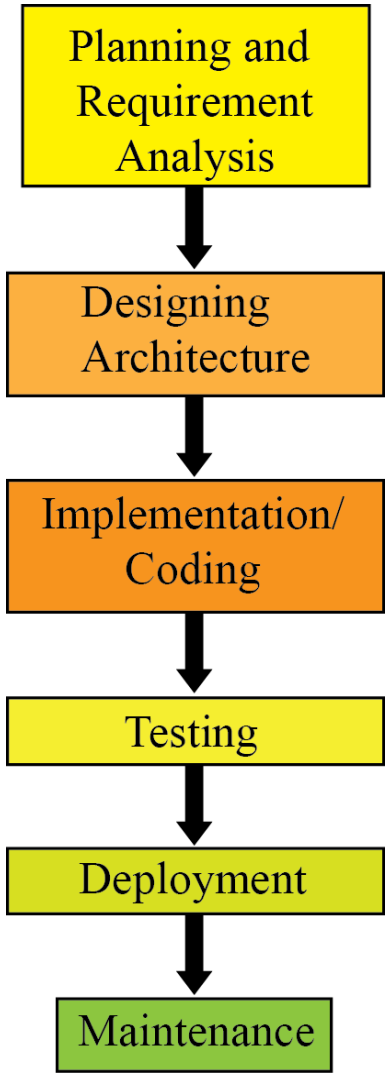


Introduction



System Implementation: Software

Software Development Process



Resource planning, schedule, functional and technical definition, documentation

Multiple architectural choices considered: layering, modules, responsibilities, implementation technologies, etc.

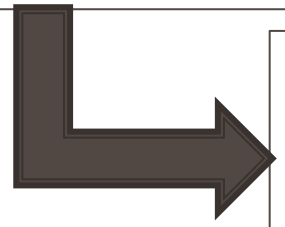
Writing actual code and building executables: compilers, interpreters, debuggers, etc. are employed

Ensuring software works as per requirements: manual and automated testing

Deployment: software released, installed in end-users environment and integrated with other parts

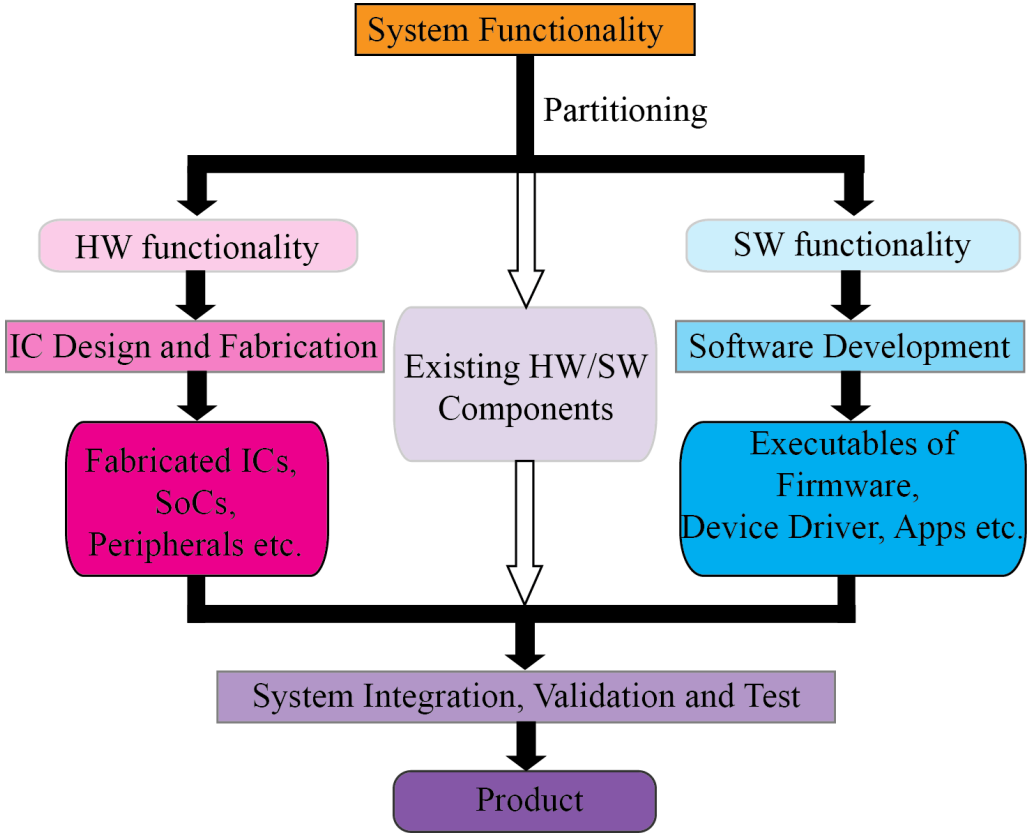
Fixing bugs, making enhancements as per need, and providing software support

Introduction



System Implementation: Integration

System Integration and Validation



- Integration performed when components become available

- Integration testing: issues of integration discovered

Validation:

- More exhaustive functional verification: fabricated chip with software (OS and actual applications) running at full speed can cover many untested scenarios
- Electrical and other issues (crosstalk, IR drop in the power supply lines, thermal effects, and process-induced variations) discovered
- Stress testing

References

- Saurabh, S. (2023). Introduction to VLSI Design Flow. Cambridge: *Cambridge University Press*.